Types and Programming Languages – Notes

goldenstack

2025-12-23

Contents

1. Introduction 2
   1.1. Types in Computer Science 2
   1.2. What Type Systems Are Good For 2
   1.3. Type Systems and Language Design 3
   1.4. Capsule History 3
   1.5. Related reading 3
Select References 4

1. Introduction

1.1. Types in Computer Science

Formal methods vary from powerful but cumbersome (Hoare logic, denotational semantics) to modest yet lightweight (type systems). The textbook focuses more on the latter variation, type systems. Type systems are generally either more practical or more abstract; once again, the textbook focuses on the more pragmatic aspect, practical type systems.

The definition of a 'type system' is rather hard to pin down. An example:

> *A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute.*

Type systems are typically used to *statically* prove information about a program (see Curry-Howard Correspondence). As with any system involving proofs, it is inherent to the system that some true statements cannot be proven. In the case of type systems, this is called being *conservative* and is represented by the type system rejecting an nonzero number of programs that do *not* display the "certain program behaviours".

The conservativity of a type system can generally be reduced by allowing more specific types, typically with a more general type system, so that certain structures and relationships can be represented more accurately. Likewise, a type system can only detect certain classes of errors; thes classes can only be expanded by allowing more generality in the type system.[1] As type systems grow to support more extended static checking methods, they approach being a proof checker.

1.2. What Type Systems Are Good For

Detecting errors: A proof system running automatically on your code allows for many errors to be detected automatically, hastening correctness.

Abstraction: Type systems can represent contracts that must be upheld, allowing more modularity by virtue of relying on contracts (types), not definitions.

Documentation: Types also act as a form of documentation in that they desribe an abstracted form of your code; importantly, your code and its types cannot become desynchronized.

Safety: Although safety is hard to define, a language is often considered 'safe' if it protects its own abstractions well with some sort of static or dynamic type safety. Several definitions of safety:

- Protecting its own abstractions (if the program is unsafe, it may violate its abstractions)
- Preventing untrapped errors at runtime (if the program is unsafe, errors may propagate)
- Completely defined by the manual (i.e. no undefined behaviour)

Efficiency: Type checkers provide more compile-time guarantees, allowing for more optimizations, such as which instructions are used or which runtime checks can be omitted.[2]

Other uses:

Security: Unsafe languages can be leveraged in ways that harm security (see Microsoft on memory safety). Type systems that prevent this can help increase the security of programs.

[1] Generality is not always the best term to describe this behaviour. For example, supporting the access of private data emitting an error requires arguably a more specific type system, to handle an invented feature.

[2] I do take issue with this statement. The point at which one has the most information about program functionality and tendencies is at runtime; thus, there is a higher performance ceiling when concerned with runtime checks (see Itanium as a prominent pro-runtime/pro-JIT example). Furthermore, a sufficiently intelligent compiler can hypothetically determine all information that a type-checker would be able to—it is simply much more difficult to write a compiler as such. In a way, efficiency *does* apply here—just as compiler development efficiency instead of program runtime efficiency. I digress.

Data storage: Types are on a rise in database or data storage systems, with JSON schemas and `.xsd` files as just two examples. They help structure data, and have all the same benefits types do normally.

1.3. Type Systems and Language Design

Type systems, like any other feature, should not be bolted on to an existing language. It is best when all features are designed hand-in-hand. This is exemplified for type systems specifically as they are often core features of a language, around which syntax, semantics, and functionality are defined.

Note that the importance of a type system to any particular language is unrelated to the two separate problems of:

- Having dynamic (Python, etc.) versus static typing (C, etc.).
- Having implicit (ML, etc.) versus explicit type annotations (Java, etc.).

1.4. Capsule History

In the early to mid 1950s, type systems were first used to distinguish between integer types and floating-point types. In the late 1950s and early 1960s, this was extended to include basic structured data (such as arrays of records) and higher order functions. Later, in the 1970s, these were expanded into richer concepts like abstract data types, modules, subtypes, and parametric polymorphism. Around this time, computer scientists began to draw parallels between these type systems and more formal mathematical logic.

The original textbook has an extended timeline. A few select choices[3]:

- 1900s: Formalization of mathematics
- 1930s: Untyped lambda calculus
- 1960s: Curry-Howard Correspondence
- 1970s: Intuitionistic type theory, ML, and ADTs
- 1980s: Linear logic, the calculus of constructions, dependent types, and effect systems.
- 1990s: Typed assembly

The textbook was produced in 2002 and thus has nothing past the 1990s.

1.5. Related reading

- *Foundations for Programming Languages* by Mitchell (1996)
- *Theories of Programming Languages* by Reynolds (1998)
- *The Structure of Typed Programming Languages* by Schimdt (1994)
- *Basic Simple Type Theory* by Hindley (1997)
- *Basic Category Theory for Computer Scientists* by Pierce (1991)
  - More category theory: Jacobs (1999), Asperti and Longo (1991), Crole (1994)
- *Proofs and Types* by Girard, Lafont, and Taylor (1989)
- *Computation and Deduction* by Pfenning (2001)
- *Type Theory and Functional Programming* by Thompson (1991)
- *Constructive Foundations for Functional Languages* by Turner (1991)

[3]Many important advancements have been removed. This is a select choice based on personal views.

Select References

References from *Types and Programming Languages* that seem interesting on an initial look.

- *Essentials of Programming Languages* by Friedman, Wand, and Haynes (2001)
- *Programming Language Pragmatics* by Scott (1999)